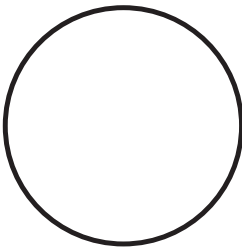


4

STRONGLY TYPED MATH



Now that we've covered the basics of functional programming, we can start connecting functional programming and math in earnest. In this and the following chapters, you'll learn to model mathematical objects with the F# type system, and we'll write functions to do computations with those mathematical objects.

Our focus in this chapter is on implementing numbers and arithmetic without using the built-in `int` and `float` types. This will allow us to review some basic math and practice data modeling with algebraic data types. Because we're working from scratch, we can also treat our types and functions as mathematical definitions and prove some facts about arithmetic from them.

The first type we'll implement will represent the *natural numbers*, which are the nonnegative integers 0, 1, 2, 3, and so on. Using our type, we'll implement addition, subtraction, multiplication, and division from scratch. After that, we'll build types representing *integers*, which include negative numbers, and *rational numbers*, which include fractions. Finally, we'll discuss the *real numbers*, a larger set of numbers obtained by filling in gaps in the rationals.

Natural Numbers from Scratch

The natural numbers are the familiar sequence of objects starting with 0, 1, 2, 3, ... and continuing without end. In F#, we can represent the natural numbers up to 2,147,483,647 with the `int` type. (F# has a similar number of negative `int` values, but these aren't natural numbers.) F# also has a `bigint` type for representing arbitrarily large whole numbers.

Our focus in this section is on creating an F# type that represents natural numbers in a much different way than `int` or `bigint`. It won't be efficient or practical for computations, but it will serve as a useful definition of natural numbers, which will allow us to concisely define many more mathematical concepts.

We'll base our type on the following intuitive properties of the natural numbers called the *Peano axioms*, which most mathematicians consider the official specification of the natural numbers.

The Peano Axioms

The natural numbers are a collection of objects satisfying the following properties:

- P0. There's a special natural number *zero*.
- P1. For each natural number, there's another natural number called its *successor*.
- P2. Zero is not the successor of any number.
- P3. Every natural number other than zero is the successor of some natural number.
- P4. If two natural numbers are distinct, they have distinct successors.
- P5. If we have a collection of *some* natural numbers that includes successors of all its members, as well as the number zero, then it is in fact the collection of *all* natural numbers.

Definitions of addition and multiplication are often included in the Peano axioms as well; we'll explore these in the following section.

The Peano axioms don't say what a natural number *is*. For example, they don't specify that a natural number is a word, or a string of digits, or any other kind of entity. Rather, they say how natural numbers *behave*. Namely, the properties imply that the natural numbers fit into an infinite sequence beginning with the special starting value, 0.

Inspired by the Peano axioms, let's use the following `Natural` type to represent natural numbers:

```
type Natural =
    | Z
    | S of Natural
```

The case `Z` represents 0, and the case `S` is a function `Natural -> Natural` returning the successor of a given value.

The simplest `Natural` value is the special zero value, `Z`. The successor of `Z` is `S(Z)`, which represents 1; the successor of 1 is `S(S(Z))`, which represents 2;

and so on. With a large enough text file, you could, in principle, represent any natural number with enough applications of S.

The values of the Natural type obey the Peano axioms: P0, P1, and P3 follow from the definition of the type, and P2 and P4 are satisfied because of the way structural equality works in F#. Specifically, structural equality forbids $Z = S(m)$ for any Natural value m , and if m and n are distinct Natural values, then $S(m)$ and $S(n)$ are distinct.

Axiom P5 is perhaps the most cryptic, but it is also satisfied. For example, if we have some `Natural` values including `Z`, and our collection contains all successors of its members, then it must contain `S(Z)`, and therefore also `S(S(Z))`, and so on, ultimately telling us that every `Natural` value is in the collection. Because the values of the `Natural` type satisfy the Peano axioms, we can use the type as our definition of the natural numbers.

This Natural type has marked disadvantages in practice. Even relatively small numbers are hard to write or read as Natural values, as the following example illustrates:

$$S(S(S(S(S(S(S(S(S(S(S(Z)))))))))$$

If we carefully count applications of `S`, we find this is the number we usually write as 13. To save ourselves effort, we can write functions `toInt` and `fromInt` that convert from `Natural` to `int`, and vice versa. For instance, we want `toInt (S(S(Z)))` to return 2 and `fromInt 13` to return the value just shown.

Like the List type, Natural is defined inductively, so we'll often use recursion when we write functions to work with it. Let's start by implementing the `fromInt` function:

```
let rec fromInt (i:int) =
  if i > 0 then
    S(fromInt (i-1))
  else
    Z
```

Here, we make use of the fact that `fromInt n` should be the successor of `fromInt (n-1)`, for `n` greater than 0. You can implement `toInt` in Exercise 4.1, using the principle that for any `Natural` value `n`, `toInt (S(n))` should equal `1 + toInt n`.

From here on, we'll casually switch between F# code and mathematical notation. In the latter, we'll denote zero by 0 and the successor function by s . For instance, $s(s(0))$ will mean the same thing as the F# value `S(S(Z))`. When we write a number like 4, you can think of it as shorthand for an expression like $s(s(s(s(0))))$ or its F# alternative, `S(S(S(S(Z))))`.

Exercise 4.1

Implement the function `toInt : Natural -> int`, converting a `Natural` value to the corresponding integer.

Exercise 4.2

The functions `toInt : Natural -> int` and `fromInt : int -> Natural` convert back and forth between the `Natural` and `int` types. Show that they do *not* define an isomorphism between the two types.

Exercise 4.3

Write a function `greater : Natural -> Natural -> bool` that indicates whether the second input `Natural` is greater than the first. Don't use `toInt!`. Instead, check whether the second is built from more applications of `S` than the first.

Exercise 4.4

The `Natural` type looks like a linked list storing no extra data in each node. Inspired by this observation, show that `Natural` and `unit list` are isomorphic types. That is, implement two functions `f : Natural -> unit list` and `g : unit list -> Natural` such that `f >> g` leaves any `Natural` value unchanged and `g >> f` leaves any `unit list` value unchanged.

Defining Arithmetic

Our next goal is to write arithmetic functions for the `Natural` type. We'll implement addition and multiplication as functions of type `Natural -> Natural -> Natural` that take two numbers and return their sum or product, respectively. Our functions will be written recursively in terms of `0` and the successor function, so they will define the arithmetic operations in terms of the Peano axioms.

NOTE

You may choose to organize the functions for the `Natural` type in a module of the same name. The `Natural.fsx` file in the online source code gives an example.

Addition

Let's start by implementing addition of `Natural` values. To do this, we'll use the idea that $m + n$ should be the result of applying the successor function m times to n . We can translate this to a recursive definition by considering both cases for m : either $m = 0$ or $m = s(k)$ for some natural number k .

We will handle the two cases with different addition rules. The first case is the base case, when $m = 0$, for which we define the following:

$$0 + n = n$$

The second possibility is the recursive case, $m = s(k)$. Intuitively, $s(k)$ is one more than k , so $s(k) + n$ should be one more than $k + n$. Therefore, we use this as our addition rule in the recursive case:

$$s(k) + n = s(k + n)$$

Because k is smaller than $s(k)$, we can be sure that the recursion always terminates. Here's an example that shows how to do addition by invoking the base case and recursive case rules.

Example: Computing $2 + 2$

Let's use our definitions to compute $2 + 2$. We have the following:

$$\begin{aligned}
 2 + 2 &= s(s(0)) + s(s(0)) && \text{(Definition of 2)} \\
 &= s(s(0) + s(s(0))) && \text{(Recursive case)} \\
 &= s(s(0 + s(s(0)))) && \text{(Recursive case)} \\
 &= s(s(s(s(0)))) && \text{(Base case)} \\
 &= 4 && \text{(Definition of 4)}
 \end{aligned}$$

Notably, if we define 2 to mean the expression $s(s(0))$, 4 to mean the expression $s(s(s(s(0))))$, and “+” according to the two rules, then this is a proof that $2 + 2 = 4$.

Our recursive addition procedure translates directly to F# as follows:

```
let rec add (m:Natural) (n:Natural) =
    match m with
    | Z -> n
    | S(k) -> S(add k n)
```

You can quickly confirm that adding $S(S(Z))$ to itself gives $S(S(S(S(Z))))$, showing that $2 + 2 = 4$.

For a bigger test, it's convenient to use the `fromInt` and `toInt` functions, as shown next:

```
toInt(add (fromInt 40) (fromInt 50)) // returns 90
```

This converts the numbers 40 and 50 to `Natural` values, adds them using the `add` function, and converts the output back to an `int`, resulting in $40 + 50 = 90$.

Notice, and remember for later, that the definition of addition and our F# implementation are both asymmetric: $m + n$ is computed by recursion on m , not n . For instance, we have $0 + 3 = 3$ by a single application of the base case, but showing that $3 + 0 = 3$ requires three recursive steps plus the base case. Of course, past experience tells us that $m + n$ should equal $n + m$ in general, but the two expressions mean different things. The fact that $m + n = n + m$ is something that needs to be proved; we'll do that in “Properties of Arithmetic” on page 89.

Exercise 4.5

Use the recursive definition of addition to compute $2 + 5$ and $5 + 2$ by hand. Which takes more steps, and why?

Exercise 4.6

Translate each intermediate expression in the proof that $2 + 2 = 4$ to F#. Check that all of them evaluate to $S(S(S(S(Z))))$.

Exercise 4.7

Viewing `unit list` values as natural numbers (as in Exercise 4.4), what

built-in `List` function computes their sum? How does the performance (number of steps) of this list operation depend on the lengths of the inputs?

Multiplication

Just as we implemented addition by repeated application of the successor function, we can implement multiplication as repeated addition. Again, we'll define $m \cdot n$ recursively in terms of the possibilities for m .

Intuitively, $m \cdot n$ means we add m copies of n . The first case is that m is 0, which would mean that there's nothing to add and that we define $m \cdot n = 0$. The second, recursive case is that $m = s(k)$ for some k , in which case $m \cdot n$ should be $k \cdot n$ plus one more copy of n . That is, we can define the following:

$$s(k) \cdot n = n + (k \cdot n)$$

Again, k is smaller than $s(k)$, so we can be assured this recursion terminates for all inputs.

In F#, this translates to the following multiply function for `Natural` values:

```
let rec multiply (m:Natural) (n:Natural) =
  match m with
  | Z -> Z
  | S(k) -> add n (multiply k n)
```

Just as the addition $m + n$ boils down to applying the successor function m times to n , the multiplication $m \cdot n$ is carried out by adding n with itself m times.

Exercise 4.8

Prove that $3 \cdot 2 = 6$, using the recursive definitions of multiplication and addition. (Hint: If you find you need to prove $2 + 4 = 6$ on the way, you're on the right track.)

Exercise 4.9

Write a function `power : Natural -> Natural -> Natural` that raises the second input to the power of the first. For instance, the function

```
power (S(S(S(Z)))) (S(S(Z)))
```

returns the third power of 2, and the result should be the `Natural` representing $2^3 = 8$. For any natural number n , the zeroth power n^0 is defined to be 1.

Exercise 4.10

Some mathematicians don't consider 0 to be a natural number, so in their formulation of the Peano axioms, 1 replaces 0 as the distinguished first number. Implement a type `Natural1` whose values represent the natural numbers starting with 1 (this type should have essentially the same structure as `Natural`). Implement new addition and multiplication functions called `add1` and `multiply1` that work with `Natural1` values.

Exercise 4.11

Using your code from Exercise 4.10, write a conversion function `toNat0 : Natural1 -> Natural`. We can't convert the other way in general, because there's no `Natural1` value representing 0, but we *can* add a `Natural` to a `Natural1`. Write a function `increment : Natural -> Natural1 -> Natural1` that adds a `Natural` to a `Natural1`, returning a `Natural1`.

Subtraction

Let's turn to subtraction. We can define the difference $n - m$ of natural numbers n and m to be the natural number d such that $m + d = n$, if such a d exists. For instance, $5 - 3 = 2$ because 2 is the number we add to 3 to get 5. Some natural number differences, like $2 - 7$, don't exist. Here, the problem is that 7 is greater than 2, so there's no natural number d making $7 + d = 2$. Here's a proof of a smaller example, directly from the Peano axioms.

Example: Proving that $0 - 1$ is not a natural number

Let's prove that $0 - 1$ is not a natural number. If there *were* a natural number d such that $0 - 1 = d$, then by definition of subtraction we'd have $0 = 1 + d$. Using the definition of addition, this gives us

$$0 = 1 + d = s(0) + d = s(0 + d) = s(d)$$

making 0 the successor of d . By the Peano axiom P2, 0 is not a successor, so there can be no such d after all.

You can try writing down a similar proof in Exercise 4.12.

Exercise 4.12

Use the Peano axioms and the definitions of subtraction and addition to prove that $2 - 7$ is not a natural number. (Hint: You will likely need to invoke Peano axiom P4.)

To implement subtraction for `Natural` values in F#, we need to consider the possibility that the result is undefined. Without negative numbers at our disposal, we can implement subtraction as a function that takes two `Natural` values and returns a `Natural option`. This function returns `Some` if there's a natural number result, or `None` otherwise.

The following `trySubtract` function takes inputs m and n and returns $n - m$ (that is, *the second input minus the first*):

```
let rec trySubtract (m:Natural) (n:Natural) : Natural option =
    match m,n with
    | Z,_ -> Some(n)
    | S(k), Z -> None
    | S(k), S(1) -> trySubtract k 1
```

This ordering of inputs may seem odd, but it's chosen so that a partial application like `trySubtract m` does what it sounds like: It takes an input and tries to subtract m . (Of course, you're welcome to use whatever ordering you like in your own implementation!)

The `trySubtract` function computes the difference $n - m$ in three cases. The first two are base cases, using that $n - 0 = n$ for all n , and that $0 - s(k)$ is not a natural number. You can prove both of these facts directly in Exercise 4.13.

In the final case, the inputs are both nonzero: $m = s(k)$ and $n = s(\ell)$. It uses the fact that $s(k) - s(\ell) = k - \ell$, which is intuitively true (you can try proving it in Exercise 4.50). The upshot of this case is that k and ℓ are smaller than the original m and n , meaning the recursion will reach the base case eventually and terminate.

Here's a quick test to verify that the function works:

```
> trySubtract (S(Z)) (S(S(Z)));;
val it : Natural option = Some (S Z)

> trySubtract (S(S(Z))) (S(Z));;
val it : Natural option = None
```

The `trySubtract` function tells us that $2 - 1 = 1$ but that $1 - 2$ is not a natural number.

Exercise 4.13

Use the definitions of addition and subtraction to prove that $n - 0 = n$ for any natural number n . Prove also that $0 - s(k)$ is never a natural number.

Exercise 4.14

Write an F# function

```
absoluteDifference : Natural -> Natural -> Natural
```

that returns the *absolute difference* of the two input values, or the absolute value of the difference between them. In other words, `absoluteDifference n m` should return whichever of `trySubtract n m` or `trySubtract m n` is a valid natural number.

Division

The final basic arithmetic operation is division. For natural numbers n and $m \neq 0$, their *quotient* is written $n \div m$ or n/m , and it's defined to be the number q such that $m \cdot q = n$, if such a number q exists. For instance, $10 \div 2 = 5$, which means $5 \cdot 2 = 10$. To indicate that the quotient $10 \div 2$ exists, we say 2 *divides* 10, or 10 *is divisible by* 2.

By contrast, there's no natural number q such that $2 \cdot q = 3$, so $3 \div 2$ is not defined; 2 does *not* divide 3. As a matter of definition, we don't allow division by 0; we don't give any meaning to an expression like $n \div 0$. Combining these facts, you can try to write your own function to divide one `Natural` by another. Since the quotient is not guaranteed to exist, you can use `Natural option` for the return type.

Another variant you may remember from elementary school is *division with a remainder*. For natural numbers n and $m \neq 0$, m may not divide

n , in which case there's no natural number q such that $m \cdot q = n$. In this case, there's still a largest number q such that $m \cdot q < n$, and we can write $n = m \cdot q + r$ for some r . In this context, q is still called the *quotient* and r the *remainder*. For instance, 14 is not divisible by 3, but we can write $14 = 4 \cdot 3 + 2$, where 2 is the remainder.

In the following exercises, you can implement division as well as division with a remainder.

Exercise 4.15

Implement a division function

```
divide : Natural -> Natural -> Natural option
```

that returns the quotient of the inputs, if it exists, or `None` otherwise.

Exercise 4.16

Write an F# function

```
divideRemainder : Natural -> Natural -> (Natural * Natural) option
```

that returns a pair of the quotient and remainder. In the case of division by 0, return `None`.

Properties of Arithmetic

This section shows how to prove some general properties of our arithmetic operations, using their definitions along with a proof technique called *mathematical induction*. This is a bit of a digression—you don't need to understand mathematical induction to follow the rest of the book—but it will allow us to begin our study of algebra on solid footing. The facts we prove with mathematical induction will serve as the basic moves of algebra, as we will discuss in Chapter 7.

We already know how to prove an equation like $2 + 3 = 3 + 2$, by directly applying the definition of addition. Now we want to verify more general facts, like $a + b = b + a$ for *any* pair of natural numbers a and b . Table 4-1 lists some essential arithmetic facts like this one, which are true for any natural number inputs.

Table 4-1: Basic Properties of Arithmetic for Any Natural Numbers a , b , and c

Property	Equality
Associative property of addition	$(a + b) + c = a + (b + c)$
Left identity for addition	$0 + a = a$
Right identity for addition	$a + 0 = a$
Commutative property of addition	$a + b = b + a$
Associative property of multiplication	$(a \cdot b) \cdot c = a \cdot (b \cdot c)$
Left identity for multiplication	$1 \cdot a = a$
Right identity for multiplication	$a \cdot 1 = a$
Commutative property of multiplication	$a \cdot b = b \cdot a$
Left distributive property of multiplication over addition	$a \cdot (b + c) = a \cdot b + a \cdot c$
Right distributive property of multiplication over addition	$(a + b) \cdot c = a \cdot c + b \cdot c$

You probably recognize these equalities, even if the names are unfamiliar. They serve as the basic rules of algebra, and we will make heavy use of them in Chapter 7.

Of the properties in Table 4-1, the only one we’ve proved so far is $0 + a = a$; in fact, this is part of the definition of addition, so it hardly needs a proof. We need to use mathematical induction to prove the rest of the properties.

Mathematical Induction

For a motivational example, let’s consider the right identity property of addition: $a + 0 = a$ for all natural numbers a . This makes a good first example because it’s “obviously” true, but it’s not obvious how to prove it from the definitions. Here’s a first try.

Proof attempt: The right identity property of addition

Let’s try to show $a + 0 = a$ for all natural numbers a by using the definition of addition. By the Peano axioms, either $a = 0$ or $a = s(k)$ for some natural number k . In the first case, we have $0 + 0 = 0$, which is true by the base case of addition. In the second case, we’d like to show $s(k) + 0 = s(k)$. By the recursive case, we have this:

$$s(k) + 0 = s(k + 0)$$

But to complete the proof, we’d need to show $k + 0 = k$ for any natural number k . We’re back where we started!

The root of the problem is that when we’re trying to prove $a + 0 = a$, we’re actually trying to prove an infinite collection of facts at once. The se-

quence of facts begins

$$\begin{array}{lll}
 \text{Fact 0:} & 0 + 0 & = 0 \\
 \text{Fact 1:} & 1 + 0 & = 1 \\
 \text{Fact 2:} & 2 + 0 & = 2 \\
 \text{Fact 3:} & 3 + 0 & = 3 \\
 \text{Fact 4:} & 4 + 0 & = 4 \\
 & \vdots & \\
 & & \vdots
 \end{array}$$

and continues infinitely, with one fact per natural number. Proving any of these individually is no challenge. Here, for instance, is a proof of Fact 4.

Proof: $4 + 0 = 4$

We use four applications of the recursive case for addition, plus one application of the base case:

$$\begin{aligned}
 4 + 0 &= s(s(s(s(0)))) + 0 \\
 &= s(s(s(s(0))) + 0) \\
 &= s(s(s(s(0)) + 0)) \\
 &= s(s(s(s(0) + 0))) \\
 &= s(s(s(s(0 + 0)))) \\
 &= s(s(s(s(0)))) \\
 &= s(s(s(0))) \\
 &= 4
 \end{aligned}$$

The key observation we can make is that the second line, $s(s(s(s(0))) + 0)$, is equal to $s(3 + 0)$. If we had already proved Fact 3, which says $3 + 0 = 3$, we could have finished the proof in one step here. It turns out this happens in general: If we use Fact n , we can easily prove the subsequent Fact $s(n)$.

Proof: Fact $s(n)$ from Fact n

Suppose we know Fact n from our list—that is, $n + 0 = n$. Then, using the recursive case of addition and Fact n , we show

$$s(n) + 0 = s(n + 0) = s(n)$$

which is Fact $s(n)$.

This principle convinces us that we *could* prove any fact in the list. Suppose we wanted to prove Fact 1,000,000. We know Fact 0 is true, since $0 + 0 = 0$. We can then use Fact 0 to prove Fact 1, Fact 1 to prove Fact 2, Fact 2 to prove Fact 3, and so on, until eventually we use Fact 999,999 to prove Fact 1,000,000. The logical jump from here is that in showing we *could* prove any of the facts, we *have* proved all of them.

The principle of *mathematical induction* is that if we have a sequence of facts where the first one is true and each fact implies the next, then indeed all the facts in the sequence are true. This follows from Peano axiom P5, as the following proof shows for the specific example of $a + 0 = a$.

Proof: The right identity property of addition

Let's complete the proof that $a + 0 = a$. Remember, Peano axiom P5 says that if we have a collection of *some* natural numbers that includes successors of all its members, as well as the number 0, then it is in fact the collection of *all* natural numbers.

Let C be the collection of natural numbers such that $a + 0 = a$. That is, C is the collection of natural numbers a for which Fact a holds. C contains 0 because $0 + 0 = 0$. If C contains any n , then C contains $s(n)$, because we showed that Fact $s(n)$ is true if Fact n is. Therefore, C is the collection of all numbers, and $a + 0 = a$ for all natural numbers.

We'll look at more examples in the next section, but it may help to adapt and apply this result in the following exercises first.

Exercise 4.17

Prove that $a \cdot 1 = a$ for all natural numbers a . (Hint: Use a modified version of our proof of the right identity property of addition.)

Exercise 4.18

Prove $1 \cdot a = a$ and $2 \cdot a = a + a$ for all natural numbers a . (If you use $a + 0 = a$, then you won't need induction for either.)

Inductive Proofs of Arithmetic Properties

Mathematical induction can be mind-bending, but in practice it's quite simple to use for proving facts about natural numbers. If we want to prove something about every natural number, we just have to show that it's true for 0 and that if it's true for some n , it's true for $s(n)$ as well.

With this technique in hand, it's possible to prove all the arithmetic properties from Table 4-1. Here are two examples, starting with the associative property of addition.

Proof: The associative property of addition

Let's prove that $(a + b) + c = a + (b + c)$ for all natural numbers a , b , and c . Our strategy is to fix b and c and use mathematical induction to prove the fact for all a .

First, we want to show the fact holds for $a = 0$. We have

$$(0 + b) + c = b + c = 0 + (b + c)$$

using the base case of addition twice.

Second, let's assume that the result holds for $a = k$ —in other words, that $(k + b) + c = k + (b + c)$ —and show that the result holds for $a = s(k)$. Indeed, we have the following:

$$\begin{aligned} (s(k) + b) + c &= s(k + b) + c \\ &= s((k + b) + c) \\ &= s(k + (b + c)) \\ &= s(k) + (b + c) \end{aligned}$$

In the first, second, and fourth lines, we use the recursive case of the definition of addition, and the third line uses our assumption that the result holds for $a = k$. By the principle of mathematical induction, the result holds for all values of a , and the proof is complete.

Here's another fact that is not in our table but is a useful intermediate result: For all natural numbers m and n , we have $s(m) + n = m + s(n)$.

Proof: $s(m) + n = m + s(n)$

We will show $s(m) + n = m + s(n)$ by induction on the variable m . That is, we'll prove the case where $m = 0$ and show that the case $m = k$ implies the case $m = s(k)$.

If $m = 0$, we have this:

$$s(m) + n = s(0) + n = s(0 + n) = s(n) = 0 + s(n)$$

Next, assume $s(k) + n = k + s(n)$. Setting $m = s(k)$, we have the following:

$$\begin{aligned} s(m) + n &= s(s(k)) + n \\ &= s(s(k) + n) \\ &= s(k + s(n)) \\ &= s(k) + s(n) \\ &= m + s(n) \end{aligned}$$

We use the recursive case for addition on the second and fourth lines, and the induction assumption $s(k) + n = k + s(n)$ on the third line. By mathematical induction, the proof is complete.

For one more example, let's prove the commutative property of addition: $m + n = n + m$ for all natural numbers m and n .

Proof: The commutative property of addition

We prove $m + n = n + m$ by induction on m . If $m = 0$, we have

$$0 + n = n = n + 0$$

by the base case and a previous result. Assuming the result holds for $m = k$, we prove it for $m = s(k)$:

$$\begin{aligned} m + n &= s(k) + n \\ &= s(k + n) \\ &= s(n + k) \\ &= s(n) + k \\ &= n + s(k) \\ &= n + m \end{aligned}$$

Note our use of the previous result to get to the last line. By mathematical induction, the proof is complete: $n + m = m + n$ for all natural numbers m and n .

In the following exercises, you can use mathematical induction to try to prove the rest of the properties from Table 4-1. The point of doing this is to give you confidence, when we start doing algebra, that all the moves are justified by the definitions.

Exercise 4.19

Prove the associative property of multiplication: $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ for all natural numbers a , b , and c .

Exercise 4.20

Prove the commutative property of multiplication: $a \cdot b = b \cdot a$ for all natural numbers a and b .

Exercise 4.21

Prove the distributive properties of multiplication over addition: $a \cdot (b + c) = ab + ac$ and $(a + b) \cdot c = ac + bc$.

From this point, we'll switch gears and move on to larger sets of numbers beyond the natural numbers, starting with the integers. We'll see how arithmetic works for our larger sets of numbers, but we won't go into all the details as we did for the naturals. For instance, it will be possible to prove from our definitions that all the arithmetic properties from Table 4-1 hold for integers as well, but we won't write out the proofs. I leave it to you to do this if you want, and you should at least convince yourself this is possible!

Integers

The *integers* are the infinite collection of numbers

$$\dots, -3, -2, -1, 0, 1, 2, 3, \dots$$

including all the natural numbers as well as a “negative” copy of each nonzero natural number. For a precise definition, the idea will be to define $-n$ as the answer to the subtraction problem $0 - n$, which had no natural number solution.

We can *almost* think of the integers as two conjoined copies of the natural numbers, one positive and one negative. We just have to be careful not to include 0 and -0 as two different values. In this section, we'll work around this issue to implement an Integer type whose values are in one-to-one correspondence with actual integer values, along with arithmetic functions to go with it.

An Integer Type

For our Integer type, we'll use the following discriminated union, including two copies of the Natural type to represent the positive and negative integers, respectively:

```
type Integer =
  | Positive of Natural
```

To avoid duplicating the value 0, we'll interpret `Positive(n)` as the natural number n and `NegativeS(n)` as the negative successor of n , that is, $-s(n)$ or $-(1 + n)$. For instance, `Positive(Z)` is 0 and `Positive(S(Z))` is $s(0) = 1$, while `NegativeS(Z)` is $-s(0) = -1$ and `NegativeS(S(Z))` is $-s(s(0)) = -2$.

The negation symbol $(-)$ means we're talking about a negative number, but we can also view the symbol as an operation that takes an integer and returns another. We define $-0 = 0$, $-(-n) = n$, and of course the negation $-(n)$ of a positive number is the negative number $-n$. Here's an F# implementation of the negation operation:

```
let negate (x:Integer) : Integer =
    match x with
    | Positive Z -> Positive Z
    | Positive(S(n)) -> NegativeS(n)
    | NegativeS(n) -> Positive(S(n))
```

From here on, I'll assume that our `Natural` functions have been placed in an F# module of the same name. (This was explained in Chapter 3. See the Chapter 4 walk-through file in the online code to see exactly how it's done for the `Natural` module.) This code organization will allow us to define some functions for `Integer` values, while using the `Natural` functions of the same name. For instance, the following is a `toInt` function that converts an `Integer` to the corresponding `int` value:

```
let toInt (x:Integer) : int =
    match x with
    | Positive n -> Natural.toInt n
    | NegativeS n -> - (Natural.toInt (S(n)))
```

The function uses our original `Natural` version, which we now call via `Natural.toInt`. This naming strategy will allow us to implement integer arithmetic without duplicating work we've already done for natural numbers.

Exercise 4.22

Write a function `fromInt : int -> Integer` that converts an F# `int` to an `Integer` value.

Exercise 4.23

For an integer n , its *absolute value* is denoted as $|n|$. If n is positive or 0, we define $|n| = n$, and if n is negative, we define $|n| = -n$. Implement a function `absoluteValue : Integer -> Natural` that gives the absolute value of an integer.

Exercise 4.24

Test that `negate >> negate` is equal to `id<Integer>`. This demonstrates the familiar property of negation, that $-(-n) = n$. It also shows that `negate` is an isomorphism from the `Integer` type to itself.

Integer Arithmetic

We've already implemented arithmetic for natural numbers, so let's extend the operations to work for negative integers. The basic principle is that adding a negative number $-n$ should mean the same thing as subtracting n .

In turn, we know how to do subtraction for some pairs of natural numbers. The only case our `trySubtract` function chokes on is a difference of the form $0 - s(k)$, which is subtracting a positive number from 0. If we define the result of $0 - s(k)$ to be the negative integer $-s(k)$, we can take the difference of any two natural numbers and return the result as an `Integer` as follows:

```
let rec subtractNat (m:Natural) (n:Natural) : Integer =
  match m,n with
  | Z,_ -> Positive(n)
  | S(k), Z -> NegativeS(k)
  | S(k), S(l) -> subtractNat k l
```

Now, if n and m are natural numbers, we can define $n + -m$ to be the integer result of $n - m$. If we similarly define $-n + m = m - n$ and $-n + -m = -(n + m)$, we get the following definition of addition for all integers:

```
let rec add (x:Integer) (y:Integer) =
  match x, y with
  | Positive m, Positive n -> Positive(Natural.add m n)
  | Positive m, NegativeS(n) -> subtractNat (S(n)) m
  | NegativeS m, Positive(n) -> subtractNat (S(m)) n
  | NegativeS m, NegativeS(n) -> NegativeS(S(Natural.add m n))
```

In the following exercises, you can implement subtraction, multiplication, and division for the `Integer` type. We won't rely on this `Integer` type in future chapters, but it's a great demonstration that we can use algebraic data types to build mathematical objects from scratch.

Exercise 4.25

Write an `Integer.subtract` function that takes two `Integer` values as inputs and subtracts the first from the second.

Exercise 4.26

Write an `Integer.multiply` function that computes the product of two `Integer` values.

Exercise 4.27

Using the `add` function for integers as your source of truth, prove that $0 + x = x$. In other words, prove that for any `Integer` value x , we have

```
x = add (Positive Z) x
```

and thus that the left identity property from Table 4-1 holds for integers. Consider proving more properties for integers if you have the patience.

Exercise 4.28

Write a function

```
divideRemainderInt : Integer -> Integer -> Integer * Natural
```

that takes integers m and n and outputs a pair of an integer quotient q and a natural number remainder r such that $n = q \cdot m + r$. The remainder should be the smallest positive number possible, specifically $0 \leq r < |m|$.

How We Usually Represent Numbers

Let's make the connection between our custom implementations of numbers and arithmetic, and the way we work with numbers in real life. Humans and computers don't usually work with numbers in terms of 0 and the successor function. Instead, we use a *place-value* system, where numbers are strings of digits 0 through 9 and the position of each digit determines its contribution to the overall number value. For instance, the number 1,905 is short for a sum of powers of 10, as illustrated here:

$$\begin{aligned} 1,905 &= 1 \cdot 1,000 + 9 \cdot 100 + 0 \cdot 10 + 5 \cdot 1 \\ &= 1 \cdot 10^3 + 9 \cdot 10^2 + 0 \cdot 10^1 + 5 \cdot 10^0 \end{aligned}$$

This system is called *base 10*, or *decimal*, because there are 10 digits, and the digits in a number contribute various powers of 10. Alternatively, we can choose any other base $b \geq 2$, where we select b symbols as digits and they contribute powers of b according to their position. The most important example is base 2, or *binary*, where the digits—called *bits* in this context—are the two symbols 0 and 1.

We sometimes use a subscript to clearly indicate the base we're working in (so 10111_2 means 10111 interpreted as a base 2 number), while omitting the subscript means we're using base 10. For example, the binary number 10111_2 is equal to the decimal number 23:

$$\begin{aligned} 10111_2 &= 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 \\ &= 16 + 0 + 4 + 2 + 1 \\ &= 23 \end{aligned}$$

I'll assume you're familiar with these ideas, so I won't belabor the topic. In a few places in the book I'll mention numbers in base 2 or other bases, so if you want a refresher, try the next exercises. A first challenge (Exercise 4.29) is to interpret an `int` list of digits like `[1;0;1;1;1]` in a given base, converting it to the `int` value it represents. For instance, viewed as a binary number, this should convert to 23, while viewed as a decimal number, it should be 10111. In base 3, it is a different number again: $10111_3 = 94$. After you can convert both ways, you can try implementing addition, subtraction, and multiplication in any base by using the elementary school techniques of carrying, borrowing, and so on. As a general hint, you may find it easier to recurse on lists of digits if you reverse them.

Lists of digits or bits are much more efficient for calculations than the `Natural` type. For example, adding two 10-digit numbers as `Natural` values

would take over 1 billion steps, but if we do the usual addition algorithm in base 10, we need to add only 10 pairs of digits. The other advantage of our digit list representation is that it can handle arbitrarily large numbers. The `int` type represents integers with exactly 32 bits, so its values are capped around 2 billion, but we can easily create larger numbers as digit lists and still calculate with them efficiently. A built-in F# type called `bigint` uses the same principle to represent arbitrarily large integers, and you can test it out in Exercise 4.36.

Exercise 4.29

Write a function `fromBase : int -> int list -> int` that takes a base b and a list of digits and returns the `int` represented by those digits as a base b number. Write another function `toBase : int -> int -> int list` that takes a base b and an `int` value and converts the `int` to base b .

Exercise 4.30

Write a function `addBase : int -> int list -> int list -> int list` that takes a base b and two lists of digits and returns their sum as a base b number.

Exercise 4.31

Following Exercise 4.30, write similar functions for subtraction, multiplication, and division (you'll have to decide how to handle results that are not natural numbers).

Exercise 4.32

What is the smallest power of 7 that has 1,000 digits?

Exercise 4.33

Viewing `false` as 0 and `true` as 1, we can interpret a `bool list` as a binary number. Implement conversion functions between `bool list` and `Natural`, showing that the types are isomorphic.

Exercise 4.34

Base 16 is called *hexadecimal*. In our `int list` representation, hexadecimal “digits” are the `int` values from 0 to 15. (This is weird, but it shouldn't break the functions you wrote in the previous exercises!) Test that you can convert from hexadecimal to binary by converting each hexadecimal digit to binary and concatenating the results. For instance, 13 in binary is 1101_2 , so `[13;0;13]` in hexadecimal should translate to `[1;1;0;1;0;0;0;0;1;1;0;1]` in binary. For what pairs of bases does this kind of conversion work?

Exercise 4.35

Use your addition function from Exercise 4.30 to write a recursive function `int -> int list` that takes a number n and returns the (decimal) digits of the number 2^n . Use $2^0 = 1$ as your base case and $2^{n+1} = 2^n + 2^n$ as your recursive case. Compute the digits of 2^{100} (something you can't do with the built-in `int` type).

Exercise 4.36

F# has a built-in `bigint` type that works with arbitrarily large integers.

You can create a `bigint` value by entering some digits followed by `I`, like `16I` for 16 or `2I` for 2. You can use the ordinary arithmetic operators (like `+` and `*`) for `bigint` values, as well as some special functions attached to the type. Compute 2^{100} via `bigint` and compare this to your answer for Exercise 4.35.

Rational Numbers

By including negative numbers, we were able to define subtraction for any pair of integers. If we further include fractional numbers, we get the *rational numbers*, a set in which any division (except by 0) is defined.

For two integers x and $y \neq 0$, we can write their quotient x/y , even if there is no such integer. We declare every such quotient to be a rational number, with x as the *numerator* and y as the *denominator*. Every integer x corresponds to a rational number $x/1$, but some rational numbers, like $1/2$, don't correspond to any integer.

A Rational Type

We can represent rational numbers in F# with a pair comprising an integer numerator and integer denominator. If you want to keep working from scratch, you can use `Integer` values, but for simplicity and efficiency, we'll use the F# `bigint` type from Exercise 4.36:

```
type Rational =
    {
        Numerator : bigint
        Denominator : bigint
    }
```

To create a `bigint` value, we end our number with the character `I`, so that the following represents the number $10/5$:

```
{ Numerator = 10I; Denominator = 5I }
```

The values of this `Rational` type are *not* in one-to-one correspondence with the rational numbers. One problem is that the `Denominator` could be zero (`0I`), giving us a `Rational` that doesn't represent a valid rational number.

Another issue is that there are many ways to represent any given rational number as a `Rational`. For instance, $10/5$ and $2/1$ should be equal as rational numbers, because they are equal as quotients of integers. In mathematics, this ambiguity is handled by redefining equality for rational numbers: We say $a/b = c/d$ if $ad = bc$. This notion of equality is not captured by structural equality of `Rational` values, so if we want to test equality of `Rational` values in F#, we need to use our own test function, like the following:

```
let equal (q:Rational) (r:Rational) =
    q.Numerator * r.Denominator = r.Numerator * q.Denominator
```

In F#, it's actually possible to overload the = sign to mean a custom equality test for a given type, but for transparency we will avoid doing so in this book. Whenever we write an equal sign in F# code, you can be assured we're talking about structural equality.

Another way to test equality for rationals is to write them in *reduced form*, with their denominators as the smallest positive integers possible. For instance, 256/192 and 20/15 both have the reduced form 4/3, while 16/-32 has the reduced form -1/2. Reduced form is a *normal form*, meaning that two integer quotients represent the same rational number if and only if their reduced forms are structurally equal. Thus, another way to test equality for Rational values is to convert them to their reduced forms and compare them using the = operator.

We can compute the reduced form of an integer quotient x/y by dividing both x and y by their *greatest common divisor*, or the largest natural number d that divides both x and y , as follows:

```
let reduce (q:Rational) =
    let sign = if q.Denominator < 0I then -1I else 1I
    let gcd = bigint.GreatestCommonDivisor(q.Numerator, q.Denominator)
    {
        Numerator = sign * q.Numerator / gcd
        Denominator = sign * q.Denominator / gcd
    }
```

By dividing both x and y by d , and multiplying both by -1 to make the denominator positive if necessary, we get the reduced numerator and denominator.

Exercise 4.37

Implement a function `gcd : int -> int -> int` that takes two integers and returns their greatest common divisor. (Hint: If d is the greatest common divisor of x and y , and r is the remainder of the division x/y , then d divides r as well. Use this fact to write an efficient recursive solution.)

Rational Number Arithmetic

Rational number arithmetic is defined by the following rules:

$$(a/b) + (c/d) = (ad + cb)/bd$$

$$(a/b) - (c/d) = (ad - cb)/bd$$

$$(a/b) \cdot (c/d) = ac/bd$$

$$(a/b) \div (c/d) = ad/cb$$

Division is defined only if $c/d \neq 0$. These aren't arbitrary rules; they are the only sensible way to define the operations if we want a/b to behave like a quotient of integers a and b and for rational numbers to satisfy all the same arithmetic rules of Table 4-1. Meditate on this now, if you like, or wait for ??, when we'll have the necessary vocabulary to explain this concisely.

These formulas translate to F# functions of the type `Rational -> Rational -> Rational`. For instance, the multiplication formula gives us

```
let multiply (q:Rational) (r:Rational) =
    {
        Numerator = q.Numerator * r.Numerator
        Denominator = q.Denominator * r.Denominator
    }
    |> reduce
```

and we reduce the result to standardize the form of the output. Adopting the convention that our `Rational` functions all return reduced results, we can safely compare their outputs via structural equality. You can implement rational number addition, subtraction, division, and more in the following exercises.

Exercise 4.38

Implement `add`, `subtract`, and `divide` functions for the `Rational` type.

Exercise 4.39

How can you tell whether a rational number a/b is greater than a rational number c/d ? Implement a function `greater : Rational -> Rational -> bool` that returns `true` if the second input is greater than the first and returns `false` otherwise.

Exercise 4.40

Write a function `power : int -> Rational -> Rational` that takes an integer power p and rational number q and returns the rational number q^p . How should negative powers be defined? (Hint: Respect the formula $q^m q^n = q^{m+n}$ for all rational numbers q and integers m and n .)

Irrational and Real Numbers

The next logical steps on our tour of numbers are the irrational and real numbers. To get some intuition for them, Figure 4-1 shows some rational numbers between 0 and 1 plotted along a line (the *number line*), as if they were points on a ruler, measuring distance.

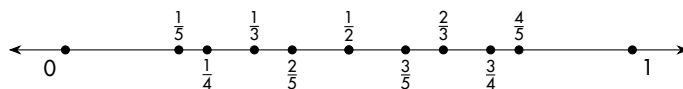


Figure 4-1: Some rational numbers plotted on the number line

If we plot a collection of rational numbers like this, gaps will always exist between them, and we can find more rational numbers in these gaps. For example, in Figure 4-1, we could add $2/9$ between $1/5$ and $1/4$, or $5/8$ between $3/5$ and $2/3$, and so on. We could continue this process as long as we wanted; we would always be able to zoom in further, find a gap, and add another rational number there.

The remarkable fact is that even the infinitely many rational numbers between 0 and 1 don't cover every point on the line. The missing points are *irrational numbers*, which by definition are not rational in that they can't be expressed as a ratio of integers. This means they can't be expressed by a finite or repeating decimal expansion. Famous examples of irrational numbers include π , e , and $\sqrt{2}$.

The irrational numbers are strange and nonintuitive, but we need to add them to the rationals to get the *real numbers*, which label all points on the continuous number line. Using pairs of real numbers, we can label all the points on a plane, and using triples of real numbers, we can label all the points in three-dimensional space. For this reason, real numbers are a good choice of number system for geometry, and we'll use them throughout the next volume. Now, let's introduce the real numbers and explore some of their peculiarities.

Looking for the Square Root of 2

To see why we need irrational numbers, consider the problem of finding the square root of 2: a positive number r such that $r^2 = 2$. We can prove that there's no *rational* number r with this property, so if we are to define $\sqrt{2}$, it will be irrational.

Proof: $\sqrt{2}$ is not a rational number

Let's suppose, temporarily, that $\sqrt{2}$ is a rational number and see what issues arise. If $\sqrt{2}$ is a rational number, it can be written in reduced form as p/q , meaning p and q are integers with no common divisor greater than 1. Then, by definition

$$(p/q)^2 = p^2/q^2 = 2$$

and therefore $p^2 = 2q^2$. Since p^2 is twice q^2 , p^2 is an even number. The square of an odd number is odd, so p^2 being even means p is even. Thus, p is divisible by 2, meaning p^2 is divisible by 4.

If the left side of $p^2 = 2q^2$ is divisible by 4, the right side must be as well. That means q^2 is divisible by 2, so q is divisible by 2. Here is the problem! We assumed that p/q was in reduced form, but now we've seen that p and q are both divisible by 2. This is a contradiction, so we conclude $\sqrt{2}$ can't be rational after all.

Even though there's no rational number equal to $\sqrt{2}$, we can see exactly where $\sqrt{2}$ *ought* to be on the number line. As the rational numbers increase, so do their squares; $1/2$, 1, $5/4$, and $4/3$ all have squares less than 2, but then $3/2$, $8/5$, 2, and every larger rational has a square greater than 2. It seems $\sqrt{2}$ is hiding somewhere in the middle, and to locate it precisely, we can look for rational numbers a and b that are very close together, such that $a^2 < 2 < b^2$.

Let's try some pairs of rational numbers whose denominators are powers of 10, meaning we can express them in decimal form. For instance, $a = 1.4 = 14/10$ and $b = 1.5 = 15/10$ lie on either side of $\sqrt{2}$, because $1.4^2 = 1.96$

and $1.5^2 = 2.25$, so $a^2 < 2 < b^2$. By trial and error, we can find a narrower and narrower window around $\sqrt{2}$, as shown in Table 4-2.

Table 4-2: Rational Number Pairs a, b , Where $a < \sqrt{2} < b$

a	b	a^2	b^2
1.4	1.5	1.96	2.25
1.41	1.42	1.9881	2.0164
1.414	1.415	1.999396	2.002225
1.4142	1.4143	1.99996164	2.00024449
1.41421	1.41422	1.9999899241	2.0000182084
1.414213	1.414214	1.999998409369	2.000001237796

All these computations can be checked with the `Rational` type its multiply operation. For example, the last value of a is the rational number

$$1.414213 = 1414213/1000000$$

and we can get a^2 by representing a as a `Rational` and squaring it.

The last values of a and b are only one-millionth ($1/1,000,000$) apart, which is a tiny interval. If the number $\sqrt{2}$ exists, we've come very close to finding it. As we go down the list of a values, we add a digit on the right each time, making the value of a^2 closer and closer to 2. No matter how many digits we add, or what base we use, we'll never get $a = \sqrt{2}$, because we've proven $\sqrt{2}$ is not rational. Even so, we can continue the sequences of a and b values, zooming in as closely as we like on where $\sqrt{2}$ should be.

So what *is* $\sqrt{2}$, if not a rational number? We can imagine Table 4-2 extended infinitely, and then we define $\sqrt{2}$ to be a new number that is larger than every a value and smaller than every b value. As we continue the a sequence, we add one digit each time, and we keep getting closer and closer to $\sqrt{2}$. Thus, we call this sequence of digits the *decimal expansion* of $\sqrt{2}$.

Exercise 4.41

Using trial and error with `Rational` arithmetic, continue the sequences of a and b values in Table 4-2 until each has 10 digits after the decimal point.

Exercise 4.42

By trial and error, find the first nine digits after the decimal point for the decimal expansion of $\sqrt{2}$. (Hint: This is the same as finding the largest integer n such that $n/1,000,000,000$ has a square less than 2.)

Exercise 4.43

Write a function `approximateSqrt2 : int -> Rational` that takes an integer power p and returns the largest rational number of the form $n/10^p$ whose square is less than 2. Use this function to get the first 1,000 digits of the decimal expansion of $\sqrt{2}$.

If you do Exercise 4.43, you'll find that the square root of 2 is approximately the following value:

```

1.41421356237309504880168872420969807856967187537694
80731766797379907324784621070388503875343276415727
35013846230912297024924836055850737212644121497099
93583141322266592750559275579995050115278206057147

```

That's 200 digits after the decimal point.

Defining Real Numbers as Dedekind Cuts

We defined $\sqrt{2}$ as a cutoff point among the rationals. While $\sqrt{2}$ is not a rational number itself, we could tell whether any given rational number was above or below it. It turns out that both rational and irrational numbers can be thought of as cutoff points for rational numbers. Cutoff points among the rational numbers are called *Dedekind cuts*, and they give us one way to define the real numbers.

Here's a brief, informal description of how this works. Let's define a real number x as a well-defined cutoff point among the rational numbers, in the sense that we can take any rational number r and say whether $r < x$. We say two cutoff points x and y are the same if $r < x$ whenever $r < y$.

As an example, $\sqrt{2}$ is a real number: We define $a < \sqrt{2}$ if a is negative or if $a^2 < 2$. Every rational number x is also a real number. It defines a cutoff point, where $r < x$ has its usual meaning.

A real number x , viewed as a cutoff point, is a machine that takes a rational number r and returns whether $r < x$ is true or false. As such, it's thought-provoking to define the type

```
type Real = Rational -> bool
```

where each real number would be represented by the function returning true for all the functions below it. Indeed, every real number can be viewed as a function that takes a rational number and returns a Boolean value.

This is a cool functional perspective, but it's not so useful in practice. Not every function `Rational -> bool` defines a cutoff point, and it turns out that for most real numbers, we can't write down their cutoff function explicitly. Even if we could represent real numbers as cutoff functions, it's not clear how to do arithmetic with them. (Can you figure out how to add or multiply two values of the `Real` type we just defined, in general?)

Despite its impracticality, the `Real` type is a nice bookend to our project of constructing number systems in F#. We implemented natural numbers and integers as discriminated union types, rational numbers as a record type, and now real numbers as a function type, so we've used all the essential algebraic data types in F#!

Exercise 4.44

Write a function `almostOneThird : int -> Rational` that takes an integer p and returns the biggest rational number of the form $n/10^p$ that is strictly less than $1/3$. What does this tell you about the decimal expansion of $1/3$?

sion of $1/3$? Write another function `almostOne` of the same type that returns the largest number of the same form that is strictly less than 1. What does this tell you about the decimal expansion of 1 ?

Exercise 4.45

Write a function `approximateCut` : `Real -> int -> Rational` that approximates a real number as a `Rational`, to a specified number of digits. Specifically, the function should take as inputs a `Real` number, as in a `Rational -> bool` representing a valid cutoff point, as well as a number p of decimal digits. Then, it should return the largest `Rational` of the form $n/10^p$ for which the `Real` returns true.

Summary

In this chapter, we used the F# type system to implement some basic math from scratch. Namely, we created types representing various kinds of numbers and functions to do arithmetic operations with them.

We started by creating a `Natural` discriminated union to represent every natural number $0, 1, 2, \dots$ as either the special value `Z`, meaning 0 , or the successor `S(m)` of another natural number m . We wrote functions to add, subtract, multiply, and divide `Natural` values, without using the built-in `int` type or its operations. Because we worked from scratch, we were able to take our arithmetic functions as definitions and prove facts like $2 + 2 = 4$. Using the principle of mathematical induction, we then proved more general properties about arithmetic, such as the fact that $m + n = n + m$ for any natural numbers n and m . Facts like this are our basic tools in algebra.

Next, we created an `Integer` type as a discriminated union of two copies of the natural numbers, representing positive and negative numbers, respectively. We further extended the integers to define the rational numbers, which include the integers as well as fractions. Our `Rational` type was a record storing two integers, representing integer quotients of the form a/b .

Finally, we explored a more subtle collection of numbers: the irrational numbers, which can't be expressed as fractions or finite decimal expansions. Both rational and irrational numbers can be expressed as cutoff points of rational numbers, so we modeled real numbers as predicates of type `Rational -> bool`, indicating whether a rational number was above or below the cutoff.

Additional Practice

Exercise 4.46

If m and n are nonzero natural numbers, prove that $n \cdot m$ is nonzero. (Hint: Let $n = s(k)$ and $m = s(\ell)$, and show that $s(k) \cdot s(\ell)$ is a successor.)

Exercise 4.47

Define a type `Binary32` as a 32-tuple of Booleans representing a 32-bit binary number. Then implement an isomorphism from the `Binary32` type to the built-in `int` type. In other words, write conversion functions `f:int -> Binary32` and `g:Binary32 -> int` whose composition in either or-

der is an identity function. (Note: This exercise is likely quite tedious, but give it a try if you're feeling brave!)

Exercise 4.48

Write a function to add two `Binary32` values, compatible with your conversion functions from Exercise 4.47. That is, adding before or after converting to `int` should yield the same result.

Exercise 4.49

If m and n are natural numbers, show that there is at most one natural number result of $n - m$. (Hint: Suppose $k = \ell = n - m$. Use mathematical induction to show that $k = \ell$.)

Exercise 4.50

Prove that `trySubtract` is correct, using mathematical induction. In particular, you'll need to show that $s(k) - s(\ell) = k - \ell$ when either difference exists.

For the next few exercises, you can use the `int` type for numbers, or, for more of a challenge, you can use the `Natural` or `Integer` types we defined in this chapter.

Exercise 4.51

A *factor* of a natural number $n > 0$ is another natural number $d > 0$ such that n is divisible by d . Write a function `listFactors : int -> int list` that takes a natural number n and returns a list of all its positive factors. (If you choose to use `Natural` numbers, your function should have type `Natural -> Natural list`.)

Exercise 4.52

A *prime* number is a natural number $p > 1$ such that the only factors of p are 1 and p . For instance, 2 is prime because its only factors are 1 and 2. By contrast, 10 is not prime because it is divisible by 1, 2, 5, and 10. Write a function `isPrime : int -> bool` that indicates whether a given number is prime.

Exercise 4.53

Create an infinite sequence of type `int seq` in F# whose values are all the prime numbers. Its first few values should be 2, 3, 5, 7, and 11. What is the 1,000th prime number?

Exercise 4.54

Implement an `int seq` value that produces the decimal expansion of $\sqrt{2}$.

Exercise 4.55

Every natural number $n > 1$ can be written in a unique way as a product of prime numbers, called its *prime factorization*. For instance, $24 = 2 \cdot 2 \cdot 2 \cdot 3$, where 2 and 3 are prime. We consider different orderings to be the same product, so $3 \cdot 5$ and $5 \cdot 3$ are the same prime factorization of 15. Write a function `primeFactorize : int -> int list` that takes a number and produces its list of prime factors in order.

Exercise 4.56

Two natural numbers are *relatively prime* if they share no positive factors other than 1. Write a function `relativelyPrime : int -> int -> bool` that computes whether a pair of inputs are relatively prime.

Exercise 4.57

For two natural numbers m and n , their *least common multiple* is the smallest natural number ℓ such that ℓ is divisible by both m and n . Write a function `lcm : int -> int -> int` that returns the least common multiple of two input numbers.

Exercise 4.58

Show how to find the greatest common divisor and least common multiple of two numbers from their prime factorizations. For instance, your functions could take a pair of prime factorizations, represented as `int list` values, and return the prime factorization of the result.

Exercise 4.59

Define an alternative type to represent integers as follows:

```
type IntPS =
  | Z
  | S of IntPS
  | P of IntPS
```

Here, `Z` represents 0, `S` represents the successor of an integer, and `P` represents the *predecessor* of an integer. For instance, `P(Z)` is `-1`, `P(P(Z))` is `-2`, `S(S(P(Z)))` is `1`, and `P(P(S(S(S(Z)))))` is `1` as well. Write F# functions to convert an `IntPS` value to an `int`, and vice versa. Do your functions exhibit an isomorphism between `IntPS` and `int`?

Exercise 4.60

Using Exercise 4.59, write a function `equal : IntPS -> IntPS -> bool` that determines whether two `IntPS` values represent the same integer. Do this without converting the `IntPS` values to `int`.

Exercise 4.61

Using the `IntPS` type from Exercise 4.59, implement addition, subtraction, and multiplication functions of the type `IntPS -> IntPS -> IntPS`.

Exercise 4.62

Our `Rational` type uses the fact that every rational number can be written as a quotient of two integers. Similarly, every integer can be written as a difference of natural numbers. Implement a record type `NatDiff` representing a difference of `Natural` values. Write conversion functions `NatDiff -> int`, and vice versa. Also write a function `equal` that checks whether two `NatDiff` values represent the same integer, without converting them to `int` values.

Exercise 4.63

Using the `NatDiff` type from Exercise 4.62 to represent integers, imple-

ment *add* and *subtract* functions that do integer addition and subtraction.

Exercise 4.64

The *golden ratio*, written with the Greek letter φ (phi), is the special number given by $\varphi = \frac{1+\sqrt{5}}{2}$. Find the first 100 digits after the decimal point in the decimal expansion of φ .

Exercise 4.65

The number π begins 3.1415926535897932384626433 Use the following infinite formula to approximate π :

$$\frac{\pi^2}{6} = \frac{1}{1^2} + \frac{1}{2^2} + \frac{1}{3^2} + \frac{1}{4^2} + \cdots$$

Note that if you use only a finite number of terms of this sum, you get a rational number. How many terms of the infinite formula do you need to use to correctly calculate the first 25 digits of π ?